

Systems Reading Group

Hi there! This semester we're planning to start a systems reading group within the club. The name isn't decided yet, so I'll just call it "reading group" for now. People from outside the club are welcome too, both to attend meetings and to take part in the projects which grow out of the reading group discussions. More on how it'll actually run in the sections below.

What is this?

The word "systems" here refers to the internals of computer systems. In CS, there are broadly 3 different "layers" a person can work on, and each one comes with a different working mental model:

- **Algorithms/theory:** The mathematical side of CS, where most research is adjacent to questions like:
 - "Is this problem solvable efficiently by a computer?" (P vs NP-type questions)
 - "What's the provably optimal way to solve a particular problem?"
 - "What are the fundamental limits on any algorithm for this problem?"
- **Applications:** Building user-facing applications (web apps, mobile apps, CLI tools, etc.) on top of existing infrastructure (operating systems, database engines, networking stacks, etc.) while mostly treating that infrastructure as a black box.
- **Systems:** The infrastructure the applications above are built on, which includes operating systems, database engines, networking stacks, distributed consensus protocols, emulators, hypervisors, device drivers, etc. The working mental model here is "how do I make X run correctly, fast, and reliably on real hardware, possibly across many machines?" Success is measured in terms of benchmarks involving variables such as latency, uptime, and throughput.

A technical reading group is basically a group of people with similar interests who pick a topic or a book and meet regularly (biweekly, or whatever frequency works), where someone explains a paper or a portion of the book each time. The good thing is that no one person needs to fully understand a "hard" paper alone. Someone else in the room who's more familiar with that area can pick up the slack.

So the idea for this reading group is to create a space for technically curious people who like tinkering and going deep into how computer systems actually work.

Who is this for?

Anyone who is curious about how computer systems actually work under the hood and loves tinkering, regardless of prior experience. You don't need to already know about advanced distributed systems theory to hop in. The only real prerequisite is curiosity to try something unfamiliar.

How would it work?

Most reading groups follow a fixed syllabus or set of books ([CS:APP](#), [DDIA](#), etc.) that members read through sequentially and then discuss. I don't want to constrain this one that way, so it's going to be a bit looser. Every two weeks, a member presents on any of the following:

- An interesting project someone stumbled across either something solving a genuinely new problem, or an existing problem solved in an interesting way (in terms of performance, design, or approach)
- Someone's own side project, as a show-and-tell
- A paper or a conference talk ([CppCon](#), [TigerBeetle's Systems Distributed](#), [GOTO](#), etc.)

Alongside the biweekly sessions, there'll be ongoing async discussion on WhatsApp in between meetings.

The main difference between this reading group and existing software domain lectures is that the reading group is meant to be loose and explore a lot more ground. Members can be interested in different domains, and even without prior knowledge of a particular one, the reading group is a way to at least get a working understanding of it.

Good examples for an existing reading group with similar structure are [Boston Protein Design and Modelling Club](#), [ML for protein engineering seminar series](#), [Papers We Love](#).

What will be covered?

The list of topics that could be covered is quite broad. Here are a few examples to get a "taste" of what would be the vibe of the reading club. This list isn't exhaustive.

- **Programming languages:** compilers, parsers, interpreters, type systems, developer tooling
- **Operating systems:** kernels, schedulers, device drivers, filesystems, eBPF
- **Distributed systems:** consensus protocols, replication strategies, consistency models, gossip protocols
- **Databases:** query engines, transaction processing, storage formats, indexing
- **Performance:** SIMD, hardware acceleration, profiling, parallelism
- **Graphics programming:** shaders and rendering pipelines, ray tracing, GPU programming, game engines
- **Virtualization:** hypervisors, emulators, microVMs, container runtimes
- **Security:** sandboxing, cryptographic protocols, exploit development, fuzzing
- **Machine learning systems:** training infrastructure, inference engines, model serving
- **Bioinformatics:** sequence alignment, genome assembly, computational biology algorithms